

Compiler Construction

⋈ Naive Register Allocation ⋈

Goals of register Allocation

How to assign variables to finitely many registers?

What to do when it can't be done?

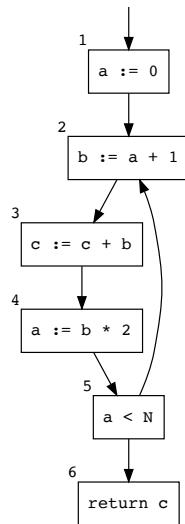
How to efficiently do so?

Goals of register Allocation

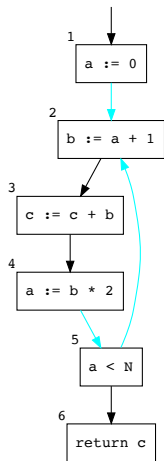
How to assign variables to finitely many registers?

Example: flowgraph

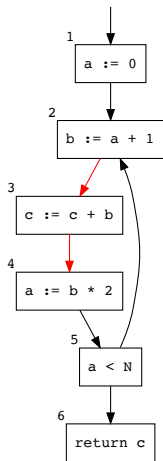
```
    a := 0
L1: b := a + 1
    c := c + b
    a := b * 2
    if a < N goto L1
    return c
```



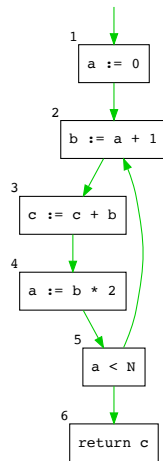
Example: liveness analysis



Liveness for a



Liveness for b



Liveness for c

Interference graph

Observation

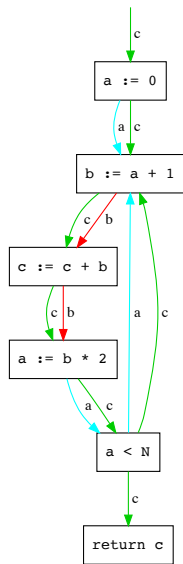
Two temporaries interfere if at some point in the program they are both live.

This means they cannot occupy the same register!

An **interference graph (IG)** is an undirected graph where:

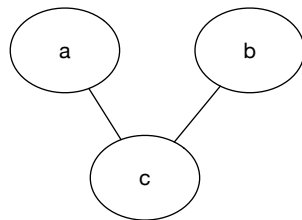
- nodes are temporaries
- there is an edge between two nodes if their live ranges overlap

How to compute interference graph?



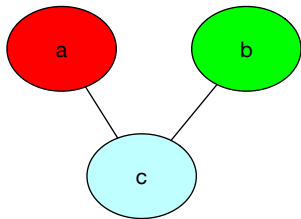
$\Rightarrow$

- Liveness analysis $\rightarrow$ liveness graph
- Depth-first walk collecting edges where several variables are live

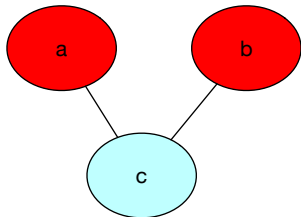


Relation between IG & register allocation

- 3 registers available R_1 , R_2 , R_3



- 2 registers available R_1 , R_3



Register allocation & graph coloring

Register allocation can be reduced to graph coloring.

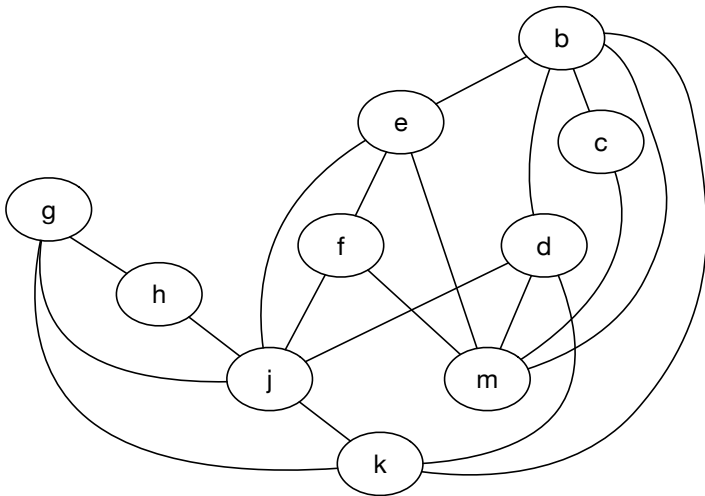
You may know that a map graph can always be colored with 4 colors... but our interference graphs are not necessarily maps!

Full example

```
# live in: k j
  g := [j + 12]
  h := k - 1
  f := g * h
  e := [j + 8]
  m := [j + 16]
  b := [f]
  c := e + 8
  d := c
  k := m + 4
  j := b
# live out: d k j
```

Full example

```
# live in: k j
g := [j + 12]
h := k - 1
f := g * h
e := [j + 8]
m := [j + 16]
b := [f]
c := e + 8
d := c
k := m + 4
j := b
# live out: d k j
```



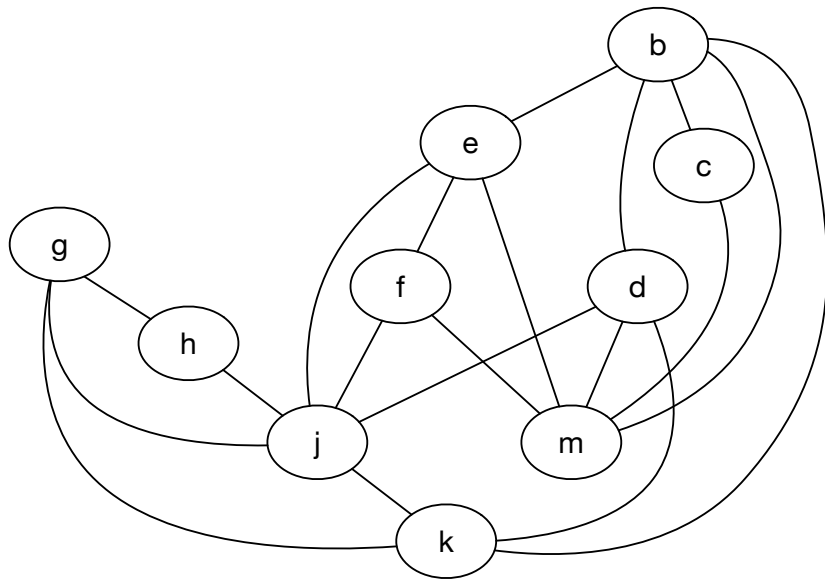
Simplification Phase

In the simplification phase, the algorithm looks for nodes with **degree less than k** , where k is the number of available registers.

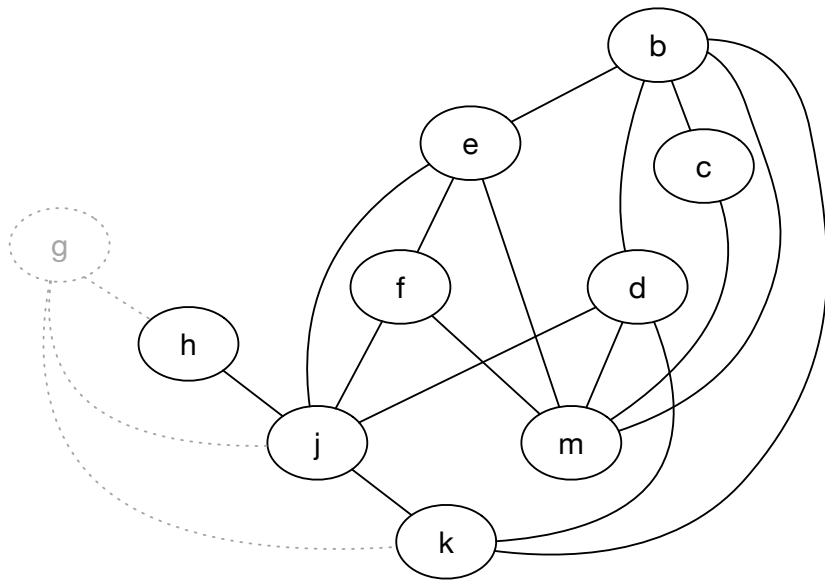
Nodes with fewer than k neighbors are considered *low-degree nodes* and are good candidates for register assignment later without conflicts.

- Such nodes are temporarily removed from the interference graph.
- Each removed node is pushed onto a stack that records the removal order.

Simplification ($K=4$)

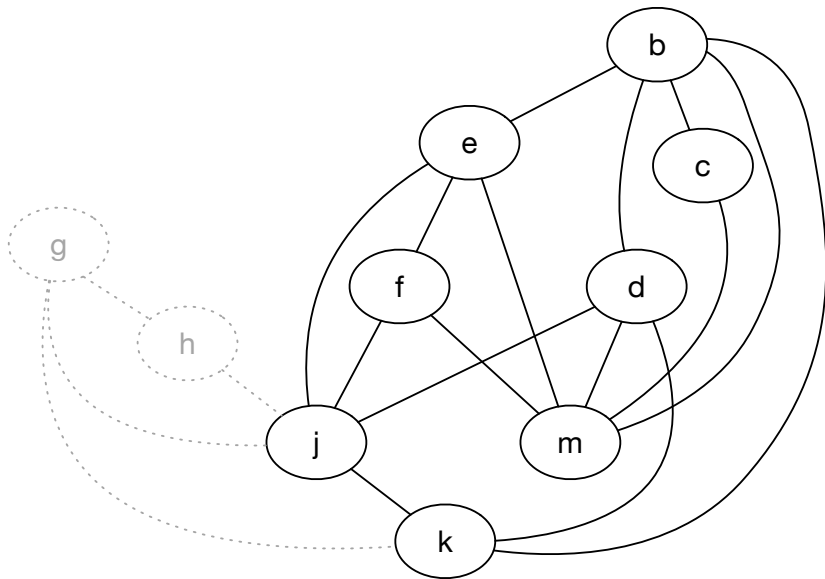


Simplification ($K=4$)



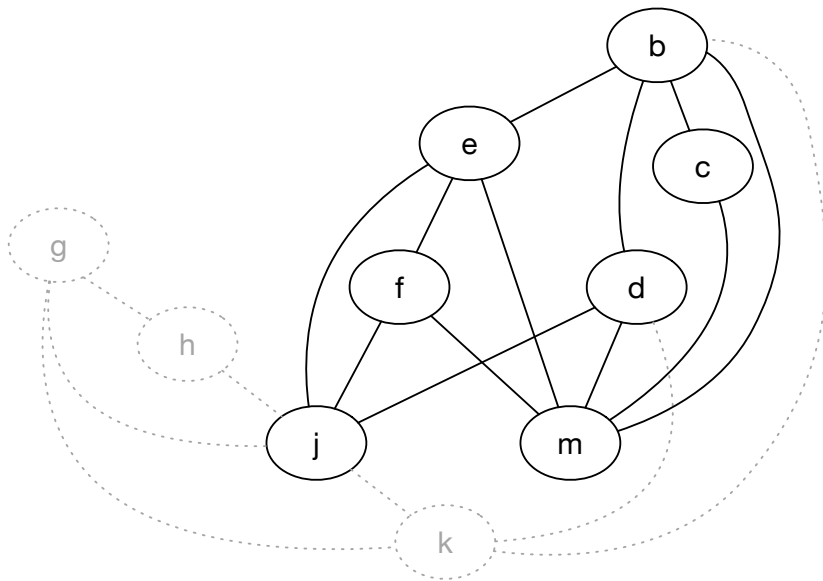
Remove g

Simplification ($K=4$)



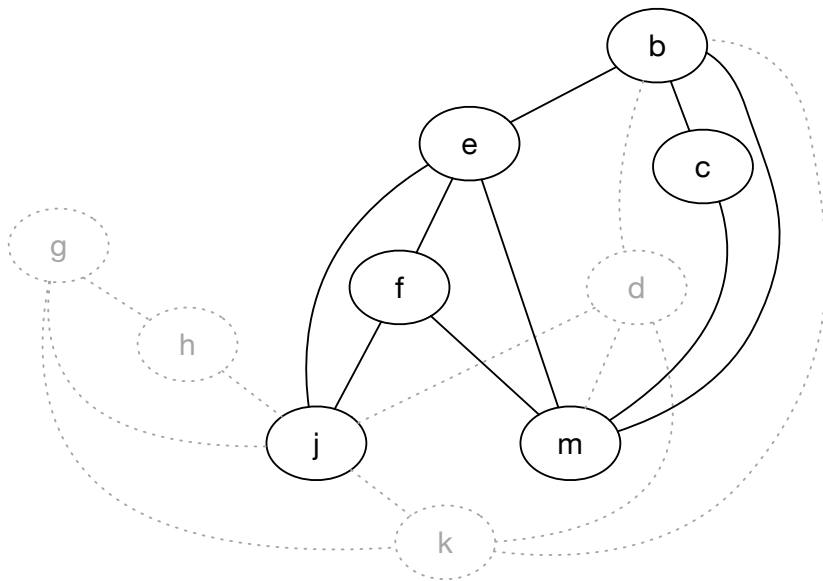
Remove h
Remove g

Simplification ($K=4$)



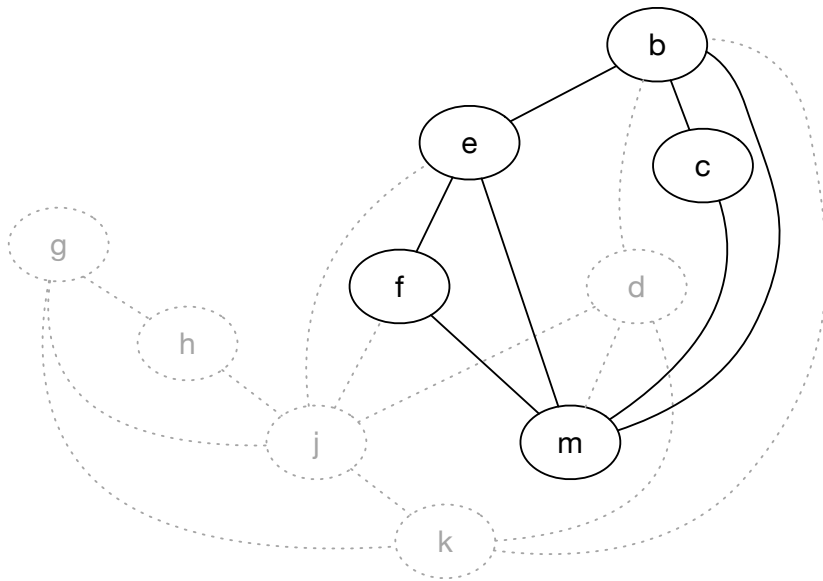
Remove k
Remove h
Remove g

Simplification (K=4)



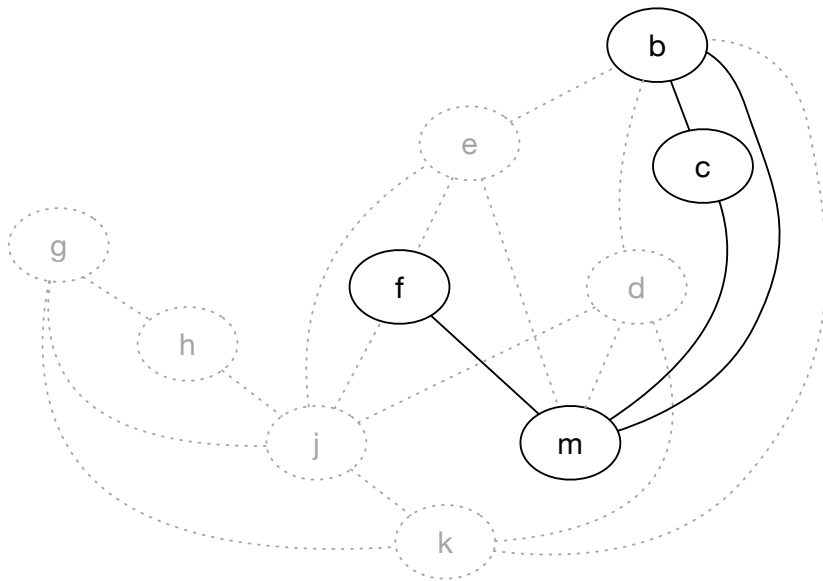
Remove d
Remove k
Remove h
Remove g

Simplification (K=4)



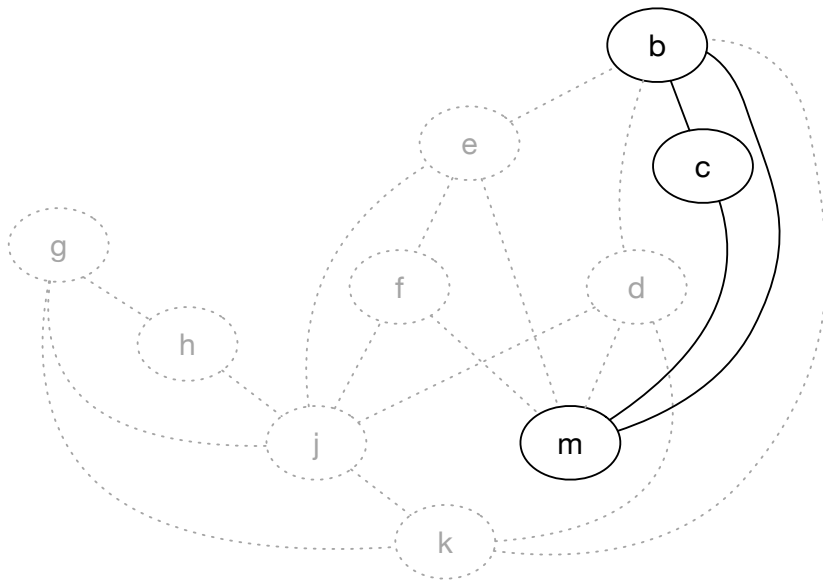
Remove j
Remove d
Remove k
Remove h
Remove g

Simplification ($K=4$)



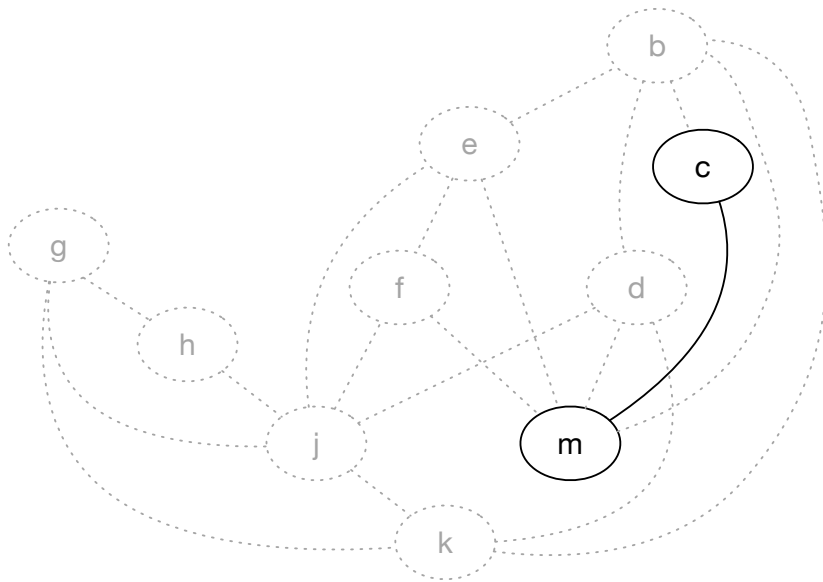
Remove e
Remove j
Remove d
Remove k
Remove h
Remove g

Simplification ($K=4$)



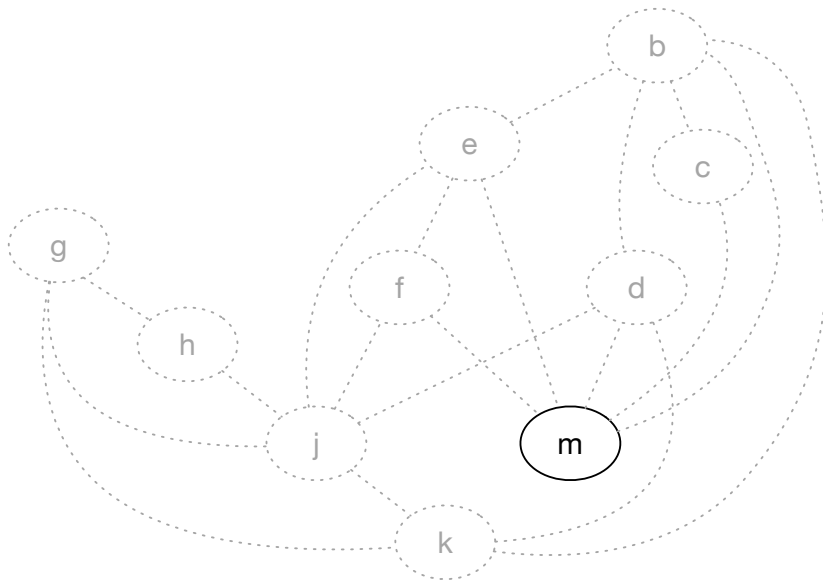
Remove f
Remove e
Remove j
Remove d
Remove k
Remove h
Remove g

Simplification ($K=4$)



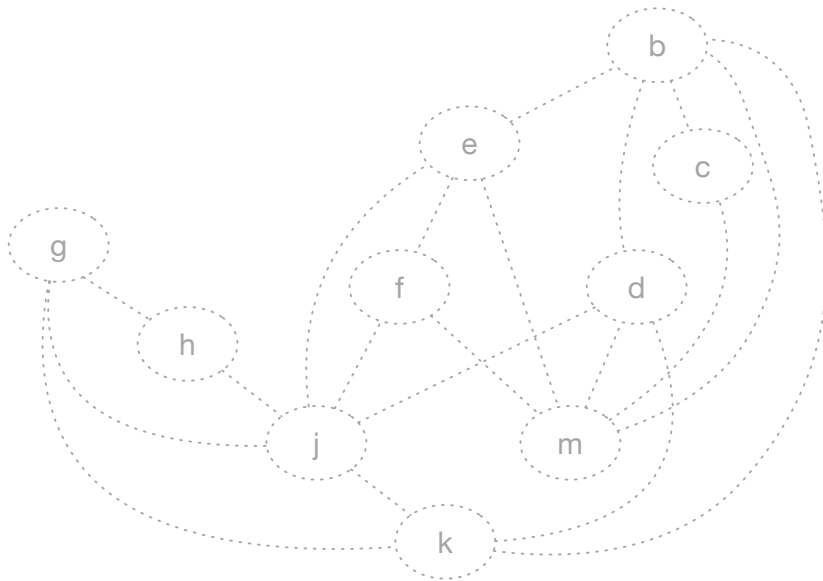
Remove b
Remove f
Remove e
Remove j
Remove d
Remove k
Remove h
Remove g

Simplification (K=4)



Remove c
Remove b
Remove f
Remove e
Remove j
Remove d
Remove k
Remove h
Remove g

Simplification ($K=4$)



Remove m
Remove c
Remove b
Remove f
Remove e
Remove j
Remove d
Remove k
Remove h
Remove g

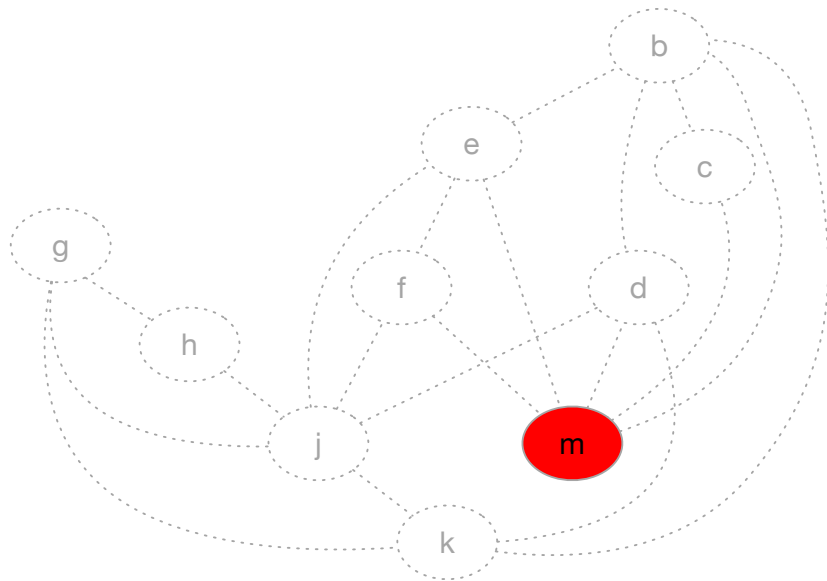
Coloring Phase

After the simplification phase, the algorithm begins the coloring phase by popping nodes off the stack in reverse order of removal.

- For each node, assign a register (color) that is not used by any of its neighbors already colored.
- Since each node was removed with fewer than k neighbors, there is always at least one available register.

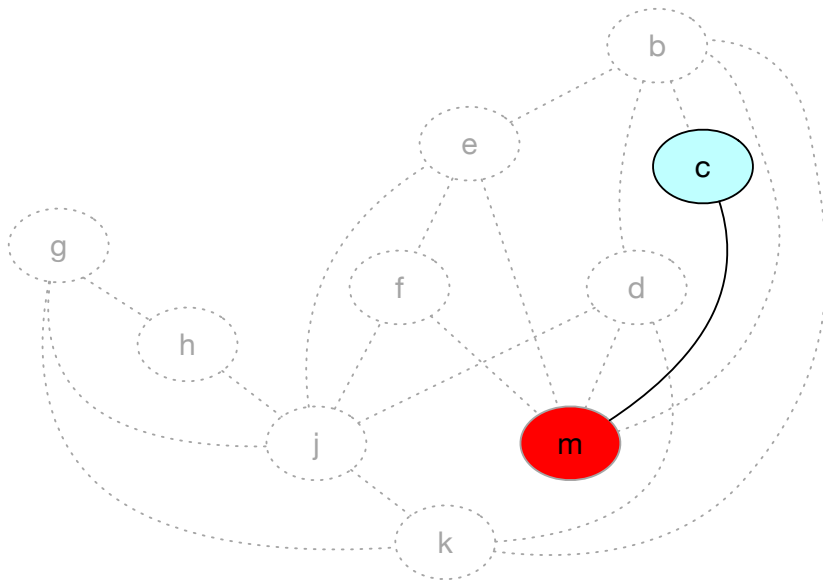
The coloring phase reconstructs a valid register assignment, if possible

Coloring



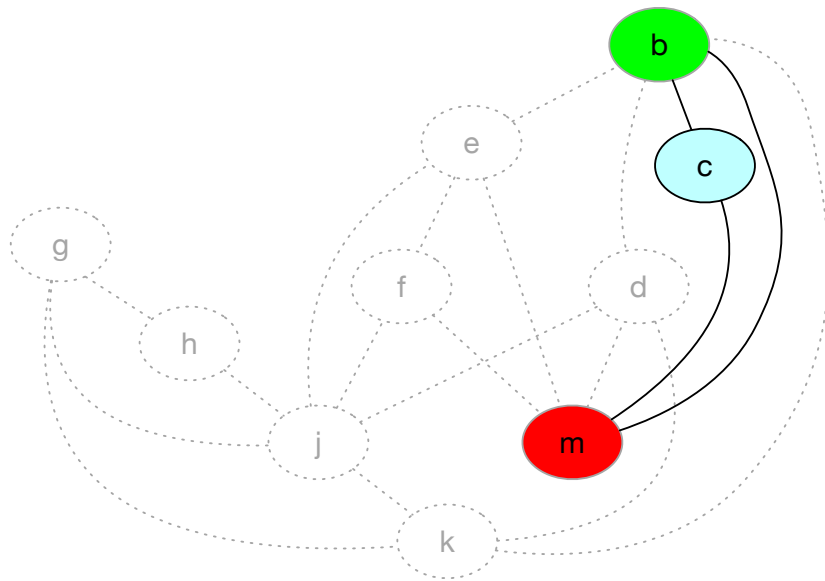
Remove c
Remove b
Remove f
Remove e
Remove j
Remove d
Remove k
Remove h
Remove g

Coloring



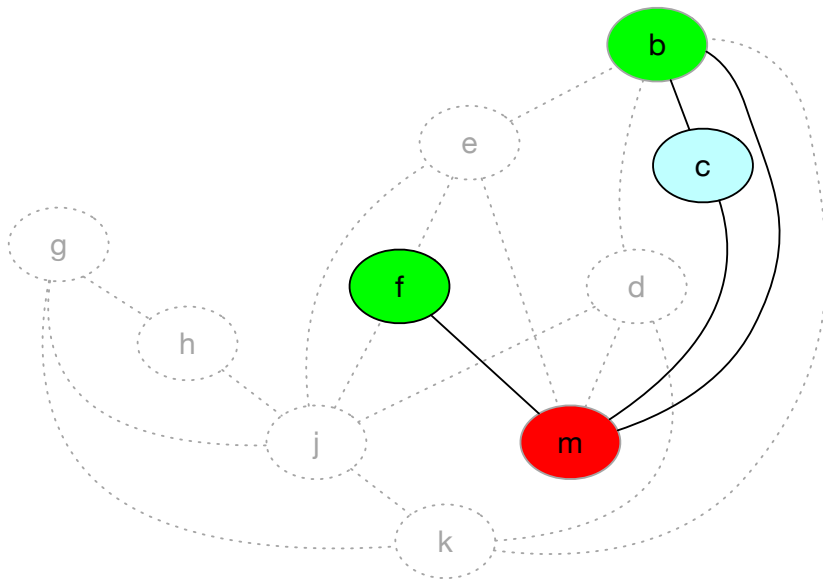
Remove b
Remove f
Remove e
Remove j
Remove d
Remove k
Remove h
Remove g

Coloring



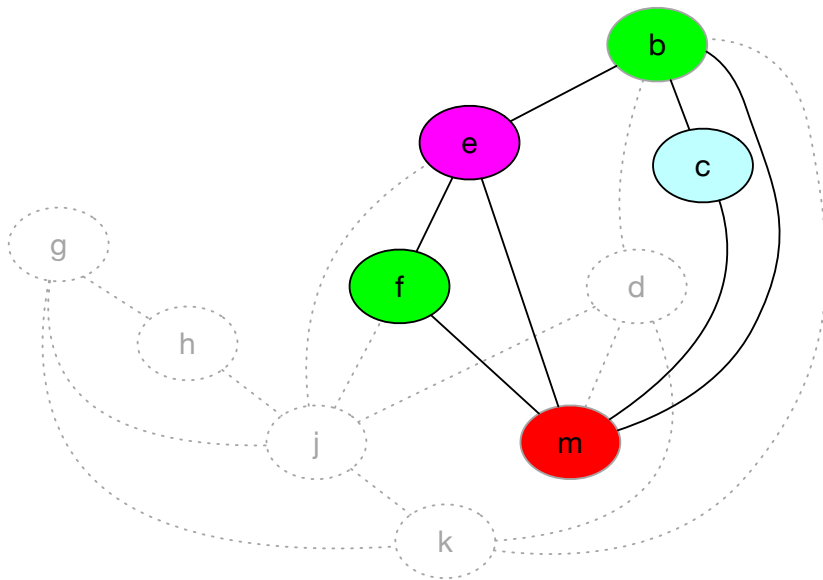
Remove f
Remove e
Remove j
Remove d
Remove k
Remove h
Remove g

Coloring



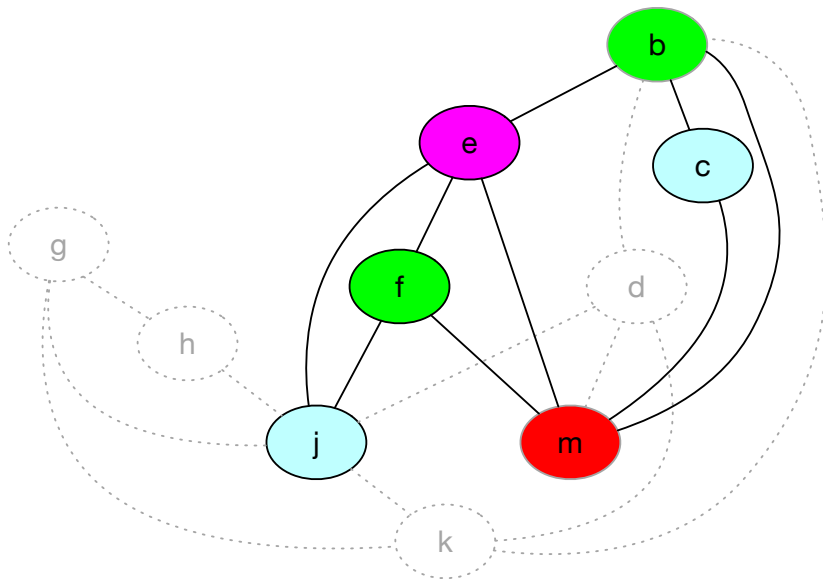
Remove e
Remove j
Remove d
Remove k
Remove h
Remove g

Coloring



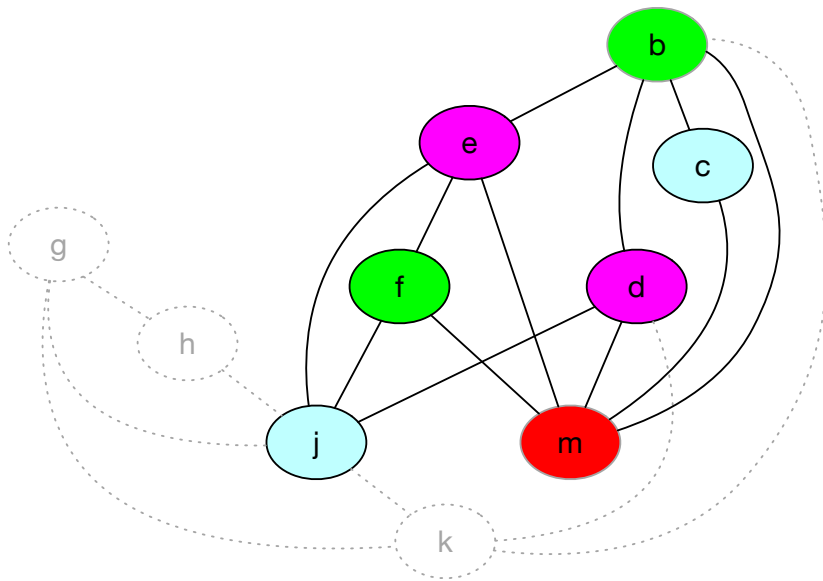
Remove j
Remove d
Remove k
Remove h
Remove g

Coloring



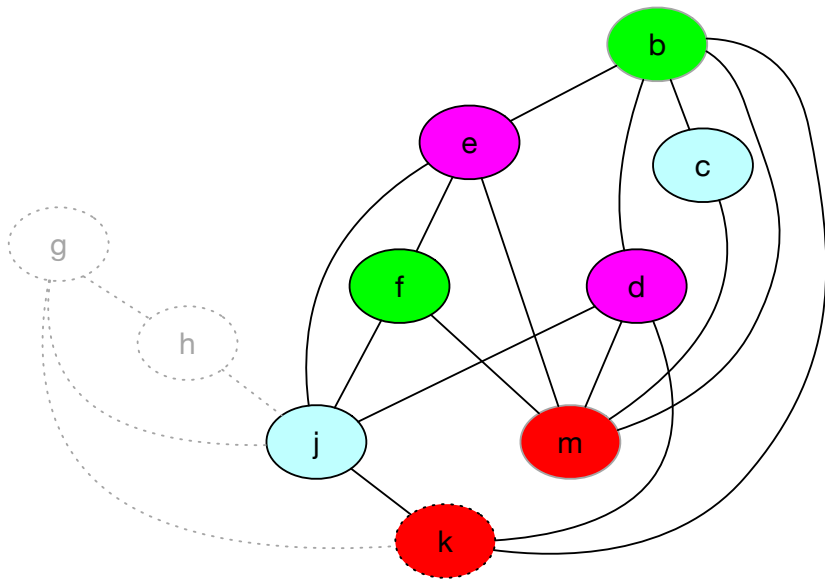
Remove d
Remove k
Remove h
Remove g

Coloring



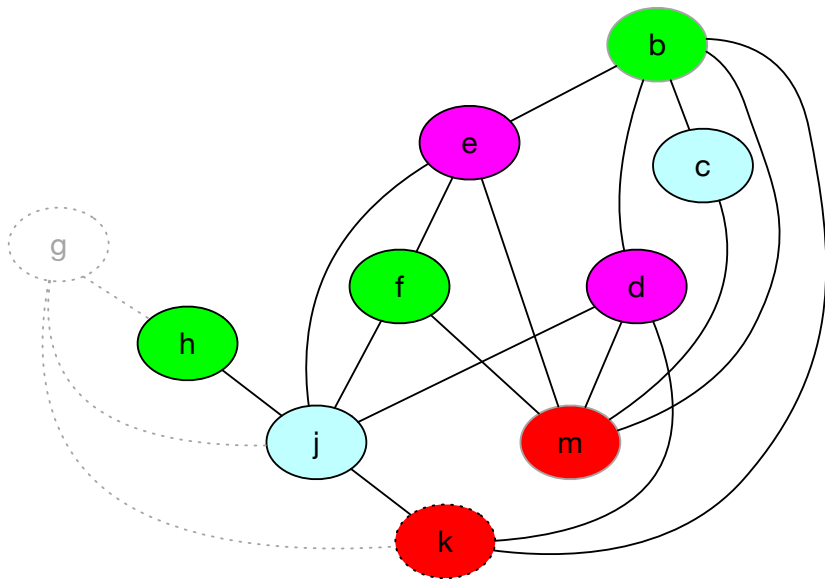
Remove k
Remove h
Remove g

Coloring



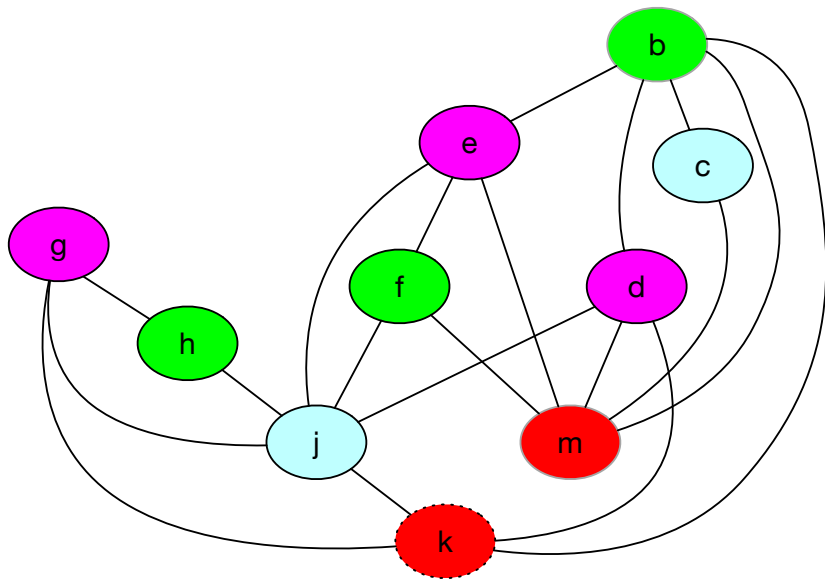
Remove h
Remove g

Coloring



Remove g

Coloring



Result

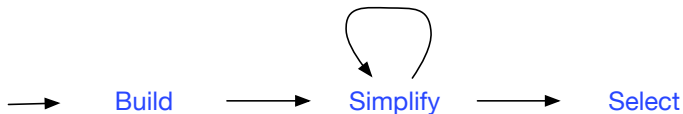
```
live in: k j
  g := [j + 12]
  h := k - 1
  f := g * h
  e := [j + 8]
  m := [j + 16]
  b := [f]
  c := e + 8
  d := c
  k := m + 4
  j := b
live out: d k j
```

```
live in: r1 r3
  r4 := [r3 + 12]
  r2 := r1 - 1
  r2 := r4 * r2
  r4 := [r3 + 8]
  r1 := [r3 + 16]
  r2 := [r2]
  r3 := r4 + 8
  r4 := r3
  r1 := r1 + 4
  r3 := r2
live out: r4 r1 r3
```

Yes, but What Color?

- Usually, first-fit (registers are ordered).
- Trying caller save first helps.
- **Biased Coloring.**
Use a color already unavailable to our neighbors.

Naive workflow



build the interference graph from the program

simplify the nodes with insignificant degree

select (or color) while rebuilding the graph.

Summary

